

PetClinic 业务与数据双服务器部署指南

本文说明如何将 PetClinic 部署为“业务服务器 + 数据库服务器”的双服务器架构。文中的 IP、账号、密码、密钥和指纹均为占位符，部署时请替换为自己的实际配置。

0. Docker 初始化

数据库服务器和业务服务器都需要先完成本节配置。以下命令适用于 Anolis OS、CentOS、RHEL 等使用 `dnf` 的 RPM 系发行版；安装方式参考 [Docker 官方 CentOS 安装文档](#)。

0.1 检查现有环境

```
cat /etc/os-release
uname -m
docker --version 2>/dev/null || true
docker compose version 2>/dev/null || true
```

如果 Docker Engine 和 Compose v2 已经正常安装，可以直接跳到“0.4 初始化 Docker 配置”。

0.2 安装 Docker Engine 和 Compose

安装仓库管理工具并添加 Docker 官方仓库：

```
dnf -y install dnf-plugins-core
dnf config-manager --add-repo \
    https://download.docker.com/linux/centos/docker-ce.repo
```

安装 Docker Engine、Buildx 和 Compose v2 插件：

```
dnf -y install \  
  docker-ce \  
  docker-ce-cli \  
  containerd.io \  
  docker-buildx-plugin \  
  docker-compose-plugin
```

如果系统已经安装发行版自带的旧版 `docker` 或 `docker-compose`，应先确认没有重要容器，再根据 Docker 官方文档处理冲突软件包。不要直接删除 `/var/lib/docker`，否则会丢失现有镜像、容器和数据卷。

0.3 启动并设置开机自启

```
systemctl enable --now docker  
systemctl status docker --no-pager
```

检查版本：

```
docker version  
docker compose version  
docker buildx version
```

验证容器运行能力：

```
docker run --rm hello-world
```

0.4 初始化 Docker 配置

创建 Docker 配置目录：

```
mkdir -p /etc/docker
```

创建或修改 `/etc/docker/daemon.json`：

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  },
  "live-restore": true
}
```

作用：

- 单个容器日志最大约 10 MB；
- 最多保留 3 个日志文件；
- Docker 守护进程短暂重启时，尽量保持已有容器继续运行。

验证 JSON 后重启 Docker：

```
python3 -m json.tool /etc/docker/daemon.json >/dev/null
systemctl restart docker
systemctl is-active docker
```

如果服务器已经有 `/etc/docker/daemon.json`，应合并现有配置，不要直接覆盖。特别是生产服务器上修改 Docker 配置前，应先执行：

```
cp /etc/docker/daemon.json \
  /etc/docker/daemon.json.backup 2>/dev/null || true
```

0.5 设置时区与部署目录

```
timedatectl set-timezone Asia/Shanghai
timedatectl

mkdir -p /opt/petclinic
mkdir -p /opt/petclinic-db
```

两台服务器分别只使用自己需要的目录：

- 业务服务器使用 `/opt/petclinic`；
- 数据库服务器使用 `/opt/petclinic-db`。

0.6 检查资源

```
free -h  
swapon --show  
df -h /  
docker system df
```

2 GB 内存的服务器建议准备约 2 GB Swap。Docker 镜像构建应在本机或 CI 中完成，服务器只负责 `docker load` 和运行容器。

0.7 镜像拉取故障说明

如果拉取镜像时出现某个第三方镜像域名无法解析，不要盲目添加网上流传的镜像源。先检查：

```
docker info  
cat /etc/docker/daemon.json  
getent hosts registry-1.docker.io
```

删除已经失效的 `registry-mirrors` 配置后重启 Docker：

```
systemctl restart docker
```

如果所在网络必须使用代理，应配置真实可用的 HTTP/HTTPS 代理，而不是长期依赖来源不明的公共镜像站。

1. 部署架构



服务器职责：

- 数据库服务器只运行 MySQL，负责持久化业务数据。
- 业务服务器只运行前端和后端容器。
- MySQL 不直接暴露到公网。
- 当前方案使用专用 SSH 隧道连接两台服务器；如果云 VPC 私网已经互通，也可以直接使用数据库私网地址。

2. 配置占位符

占位符	含义	示例格式
<BUSINESS_PUBLIC_IP>	业务服务器公网 IP	203.0.113.10
<BUSINESS_PRIVATE_IP>	业务服务器 VPC 私网 IP	10.0.1.10
<DB_PUBLIC_IP>	数据库服务器公网 IP	203.0.113.20
<DB_PRIVATE_IP>	数据库服务器 VPC 私网 IP	10.0.1.20
<SSH_USER>	SSH 登录用户	root 或普通运维用户
<DB_ROOT_PASSWORD>	MySQL root 密码	强随机密码

占位符	含义	示例格式
<DB_APP_PASSWORD>	PetClinic 数据库用户密码	强随机密码
<JWT_SECRET>	JWT 签名密钥	至少 32 字节随机值
<DEEPSEEK_API_KEY>	DeepSeek API Key	不要提交到 Git
<SSH_HOST_FINGERPRINT>	数据库服务器 SSH 主机指纹	SHA256:...
<DOCKER_HOST_GATEWAY>	业务服务器 Docker 网桥地址	通常是 172.17.0.1

3. 前置条件

两台服务器建议具备：

- Linux x86_64;
- Docker Engine;
- Docker Compose v2;
- 至少 2 核 2 GB 内存;
- 建议配置 2 GB Swap;
- 系统时区设为 Asia/Shanghai。

业务服务器安全组建议开放：

- TCP 22：仅允许运维来源 IP;
- TCP 80：HTTP;
- TCP 443：HTTPS（配置域名证书后）。

不要向公网开放：

- MySQL 3306;
- Spring Boot 8080。

4. 导出本地数据库

在 Windows PowerShell 中执行：

```
& "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysqldump.exe" `
-h127.0.0.1 `
-uroot `
-p `
--single-transaction `
--quick `
--routines `
--triggers `
--events `
--hex-blob `
--default-character-set=utf8mb4 `
--set-gtid-purged=OFF `
--no-tablespaces `
--databases petclinic `
--result-file=".\\petclinic.sql"
```

也可以使用你的数据库连接软件进行导出

不要使用项目中的 `schema.sql` 替代数据库导出文件。`schema.sql` 包含删除表的语句，适合初始化空环境，不适合覆盖已有线上数据库。

上传到数据库服务器：

```
scp -i "$env:USERPROFILE\\.ssh\<DB_ADMIN_KEY>" `
.\\petclinic.sql `
<SSH_USER>@<DB_PUBLIC_IP>:/root/
```

5. 部署 MySQL

登录数据库服务器：

```
ssh -i ~/.ssh/<DB_ADMIN_KEY> <SSH_USER>@<DB_PUBLIC_IP>
```

创建目录：

```
mkdir -p /opt/petclinic-db
mv /root/petclinic.sql /opt/petclinic-db/
cd /opt/petclinic-db
```

创建 `docker-compose.yml`：

```

services:
  mysql:
    image: mysql:8.4
    container_name: petclinic-mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: <DB_ROOT_PASSWORD>
      MYSQL_DATABASE: petclinic
      MYSQL_USER: petclinic
      MYSQL_PASSWORD: <DB_APP_PASSWORD>
      TZ: Asia/Shanghai
    ports:
      - "127.0.0.1:3306:3306"
      - "<DB_PRIVATE_IP>:3306:3306"
    volumes:
      - petclinic_mysql:/var/lib/mysql
      - ./petclinic.sql:/docker-entrypoint-initdb.d/01-
petclinic.sql:ro
  mem_limit: 768m
  logging:
    driver: json-file
    options:
      max-size: "10m"
      max-file: "3"

volumes:
  petclinic_mysql:
    name: petclinic_mysql

```

5.1 端口映射说明

```
- "127.0.0.1:3306:3306"
```

表示数据库服务器本机的 127.0.0.1:3306 转发到 MySQL 容器的 3306。SSH 隧道使用这个入口，公网无法直接访问。

```
- "<DB_PRIVATE_IP>:3306:3306"
```

表示允许云 VPC 私网内的服务器通过数据库私网 IP 访问 MySQL。若只使用 SSH 隧道，可以删除这一行。

不要使用未限定地址的：

```
- "3306:3306"
```

它通常等价于绑定 `0.0.0.0:3306`，可能让 MySQL 监听所有网卡。

5.2 数据卷说明

```
- petclinic_mysql:/var/lib/mysql
```

这是 MySQL 持久化命名卷。删除或重建容器不会删除数据库，除非执行了 `docker compose down -v` 或主动删除该数据卷。

```
- ./petclinic.sql:/docker-entrypoint-initdb.d/01-petclinic.sql:ro
```

这是只读初始化脚本。MySQL 官方镜像只会在 `/var/lib/mysql` 为空时执行它一次。数据卷完成初始化后，即使修改 SQL 文件或重启容器，也不会再次自动导入。

5.3 启动与验证

```
cd /opt/petclinic-db
docker compose up -d
docker compose ps
docker logs --tail=100 petclinic-mysql
```

验证表：

```
docker exec -it petclinic-mysql \
  mysql -upetclinic -p petclinic \
  -e "SHOW TABLES;"
```

正常情况下应看到项目所需的全部业务表。

6. 构建前后端镜像

建议在本机或 CI 构建，不要在 2 GB 服务器上运行 Maven 和 npm 构建。

```
cd <PROJECT_ROOT>
```

```
docker build --platform linux/amd64 `
  -t petclinic-backend:<VERSION> `
  .\backend
```

```
docker build --platform linux/amd64 `
  -t petclinic-frontend:<VERSION> `
  .\frontend
```

前端目录应包含 `.dockerignore`:

```
node_modules
dist
.idea
*.log
```

前端 Dockerfile 建议使用:

```
FROM node:20-alpine AS build
WORKDIR /web
COPY package.json package-lock.json ./
RUN npm ci --include=optional
COPY . .
RUN npm run build

FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /web/dist /usr/share/nginx/html
EXPOSE 80
```

打包镜像:

```
docker save `
  -o .\petclinic-images-<VERSION>.tar `
  petclinic-backend:<VERSION> `
  petclinic-frontend:<VERSION>
```

上传业务服务器:

```
scp -i "$env:USERPROFILE\.ssh\<BUSINESS_ADMIN_KEY>" `
.\petclinic-images-<VERSION>.tar `
<SSH_USER>@<BUSINESS_PUBLIC_IP>:/root/
```

7. 建立专用 SSH 数据库隧道

如果业务服务器能够直接访问 `<DB_PRIVATE_IP>:3306`，可以跳过本节并让后端直接连接私网地址。

如果私网被安全组或网络策略阻断，可使用以下 SSH 隧道方案。

7.1 在业务服务器生成专用密钥

```
install -d -m 700 /root/.ssh

ssh-keygen \
  -t ed25519 \
  -N "" \
  -C "petclinic-db-tunnel" \
  -f /root/.ssh/petclinic_db_tunnel_ed25519
```

不要把管理员私钥复制到业务服务器。隧道应使用独立密钥，便于单独撤销。

7.2 在数据库服务器限制该公钥

将业务服务器生成的公钥加入数据库服务器的 `/root/.ssh/authorized_keys`，并添加限制：

```
restrict,port-forwarding,permitopen="127.0.0.1:3306" ssh-ed25519
<TUNNEL_PUBLIC_KEY> petclinic-db-tunnel
```

这些选项表示：

- 默认禁止非必要 SSH 能力；
- 允许端口转发；
- 只能转发到数据库服务器的 `127.0.0.1:3306`。

7.3 固定数据库服务器主机指纹

从云控制台登录数据库服务器并查询：

```
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

确认输出与预期的 `<SSH_HOST_FINGERPRINT>` 一致后，在业务服务器保存主机密钥：

```
ssh-keyscan -t ed25519 <DB_PUBLIC_IP> >> /root/.ssh/known_hosts  
chmod 600 /root/.ssh/known_hosts
```

不要在未核对指纹时盲目接受新的主机密钥。

7.4 创建 systemd 隧道服务

确认 Docker 网桥地址：

```
ip address show docker0
```

通常为 `172.17.0.1`，本文用 `<DOCKER_HOST_GATEWAY>` 表示。

创建 `/etc/systemd/system/petclinic-db-tunnel.service`：

```
[Unit]  
Description=PetClinic database SSH tunnel  
After=network-online.target docker.service  
Wants=network-online.target  
  
[Service]  
Type=simple  
ExecStart=/usr/bin/ssh -NT \  
    -o BatchMode=yes \  
    -o ExitOnForwardFailure=yes \  
    -o ServerAliveInterval=30 \  
    -o ServerAliveCountMax=3 \  
    -o StrictHostKeyChecking=yes \  
    -i /root/.ssh/petclinic_db_tunnel_ed25519 \  
    -L <DOCKER_HOST_GATEWAY>:13306:127.0.0.1:3306 \  
    <SSH_USER>@<DB_PUBLIC_IP>  
Restart=always
```

```
RestartSec=5
```

```
[Install]
```

```
WantedBy=multi-user.target
```

启动并检查：

```
systemctl daemon-reload
systemctl enable --now petclinic-db-tunnel.service
systemctl status petclinic-db-tunnel.service
ss -lntp | grep 13306
```

连接链路为：

```
后端容器 host.docker.internal:13306
→ 业务服务器 <DOCKER_HOST_GATEWAY>:13306
→ SSH 到 <DB_PUBLIC_IP>:22
→ 数据库服务器 127.0.0.1:3306
→ petclinic-mysql:3306
```

8. 部署前后端

在业务服务器加载镜像：

```
mkdir -p /opt/petclinic
cd /opt/petclinic
docker load -i /root/petclinic-images-<VERSION>.tar
```

创建 `/opt/petclinic/.env`：

```
DB_USERNAME=petclinic
DB_PASSWORD=<DB_APP_PASSWORD>

JWT_SECRET=<JWT_SECRET>

AI_CHAT_MODEL=none
DEEPSEEK_API_KEY=
DEEPSEEK_BASE_URL=https://api.deepseek.com
DEEPSEEK_MODEL=<DEEPSEEK_MODEL>
```

保护配置文件:

```
chmod 600 /opt/petclinic/.env
```

创建 `/opt/petclinic/docker-compose.yml`:

```
services:
  backend:
    image: petclinic-backend:<VERSION>
    container_name: petclinic-backend
    restart: unless-stopped
    env_file:
      - .env
    environment:
      DB_URL: jdbc:mysql://host.docker.internal:13306/petclinic?
useUnicode=true&characterEncoding=utf8&serverTimezone=Asia/Shanghai&allowPublicKeyRetrieval=true&useSSL=false
      APP_DATABASE_INITIALIZE: "false"
      JAVA_TOOL_OPTIONS: -Xms128m -Xmx512m -XX:MaxMetaspaceSize=192m
    extra_hosts:
      - "host.docker.internal:host-gateway"
    ports:
      - "127.0.0.1:8080:8080"
    mem_limit: 768m
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

  frontend:
    image: petclinic-frontend:<VERSION>
    container_name: petclinic-frontend
    restart: unless-stopped
    ports:
      - "80:80"
    depends_on:
      - backend
    mem_limit: 128m
    logging:
```

```
driver: json-file
options:
  max-size: "10m"
  max-file: "3"
```

关键配置:

- `host.docker.internal:host-gateway` 让 Linux 容器能访问业务服务器宿主机。
- 后端通过 `host.docker.internal:13306` 进入 SSH 隧道。
- `APP_DATABASE_INITIALIZE=false` 防止应用启动时重新初始化或覆盖数据库。
- 后端 8080 只绑定 `127.0.0.1`, 不直接向公网开放。
- 前端 Nginx 公开 80, 并将 `/api/` 代理到 `backend:8080`。

启动:

```
cd /opt/petclinic
docker compose config --quiet
docker compose up -d
docker compose ps
```

验证:

```
curl http://127.0.0.1:8080/actuator/health
curl -I http://127.0.0.1/
curl -i http://127.0.0.1/api/owners
```

预期结果:

- 健康检查返回 `{"status":"UP"}`;
- 前端返回 HTTP 200;
- 未携带 Token 请求受保护 API 时返回 HTTP 401, 说明 Nginx 代理和后端安全链路工作正常。

9. 启用 DeepSeek AI

编辑 `/opt/petclinic/.env`:

```
AI_CHAT_MODEL=openai
DEEPSEEK_API_KEY=<DEEPSEEK_API_KEY>
DEEPSEEK_BASE_URL=https://api.deepseek.com
DEEPSEEK_MODEL=<DEEPSEEK_MODEL>
```

`AI_CHAT_MODEL` 必须写 `openai`，因为项目通过 Spring AI 的 OpenAI 兼容客户端接入 DeepSeek。

修改 `.env` 后不能只执行 `docker compose restart`，因为普通重启不会重新读取环境变量。应重新创建后端容器：

```
cd /opt/petclinic
docker compose up -d --force-recreate backend
docker compose logs --tail=100 backend
curl http://127.0.0.1:8080/actuator/health
```

不要把 API Key 写进 Dockerfile、Compose 文件、Git 仓库或聊天记录。

10. 发布新版本

业务功能更新且数据库结构未变化时，只需重新构建和替换前后端镜像。

建议每次使用新版本号，例如：

```
petclinic-backend:1.1.0
petclinic-frontend:1.1.0
```

业务服务器操作：

```
docker load -i /root/petclinic-images-1.1.0.tar

cd /opt/petclinic
cp docker-compose.yml docker-compose.yml.backup

sed -i 's/petclinic-backend:1.0.0/petclinic-backend:1.1.0/' docker-
compose.yml
sed -i 's/petclinic-frontend:1.0.0/petclinic-frontend:1.1.0/' docker-
compose.yml

docker compose config --quiet
docker compose up -d
docker compose ps
docker compose logs --tail=100 backend
```

例如新增注册功能时，如果仍复用 `sys_user`、`sys_user_role`、`owner` 和 `sys_role` 等现有表，就不需要执行数据库迁移，只更新前后端镜像。

11. 数据库结构升级

如果未来确实新增字段或表，应编写增量迁移 SQL，例如：

```
migrations/v2__add_example_column.sql
```

执行前先备份：

```
docker exec petclinic-mysql \
  mysqldump -uroot -p \
  --single-transaction \
  --routines \
  --triggers \
  petclinic \
  > /root/petclinic-before-migration.sql
```

然后执行增量 SQL：

```
docker exec -i petclinic-mysql \
  mysql -uroot -p petclinic \
  < migration.sql
```

不要通过删除数据卷或重新执行完整 `schema.sql` 升级线上数据库。

12. 回滚

发布前备份 Compose:

```
cp /opt/petclinic/docker-compose.yml \  
  /opt/petclinic/docker-compose.yml.backup
```

如果新版本异常:

```
cd /opt/petclinic  
cp docker-compose.yml.backup docker-compose.yml  
docker compose up -d
```

旧镜像确认不再需要前不要删除。

13. 常用运维命令

业务服务器:

```
cd /opt/petclinic  
docker compose ps  
docker compose logs -f backend  
docker compose logs -f frontend  
docker stats --no-stream  
curl http://127.0.0.1:8080/actuator/health  
systemctl status petclinic-db-tunnel.service
```

数据库服务器:

```
cd /opt/petclinic-db  
docker compose ps  
docker logs --tail=100 petclinic-mysql  
docker volume inspect petclinic_mysql
```

14. 故障排查

后端无法连接数据库

依次检查：

```
systemctl status petclinic-db-tunnel.service  
ss -lntp | grep 13306  
docker compose logs --tail=200 backend
```

如果直接使用 VPC 私网，则检查：

```
nc -vz <DB_PRIVATE_IP> 3306
```

前端可访问但 API 失败

```
docker compose ps  
docker compose logs --tail=100 frontend  
docker compose logs --tail=200 backend  
curl http://127.0.0.1:8080/actuator/health
```

修改 `.env` 后配置未生效

```
docker compose up -d --force-recreate backend
```

MySQL 初始化 SQL 未再次执行

这是正常行为。初始化目录只在数据卷为空时执行。对现有数据库应使用增量迁移 SQL。

15. 安全注意事项

- 不在文档、Git、聊天记录中保存真实 IP、密码、私钥、API Key 或主机指纹。
- 管理员私钥与数据库隧道私钥分开使用。
- 数据库隧道公钥只允许转发到 `127.0.0.1:3306`。
- `.env` 权限保持为 `600`。
- MySQL 不绑定公网 `0.0.0.0:3306`。

- Spring Boot 的 8080 不开放公网。
- 正式上线前修改所有演示账号密码。
- 配置域名后使用 HTTPS。
- 定期备份 petclinic 数据库，并验证备份可以恢复。